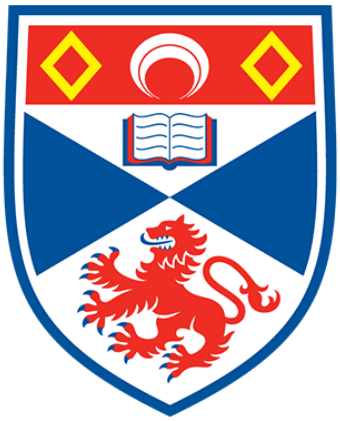
Procedural Shoot 'Em Up

Barnabas Tanczos



University of
St Andrews

Supervised by Ian Miguel

October 14, 2026

Abstract

This project involved the design and development of the shoot 'em up game *Instar*, which uses procedural generation to create a varied and interesting experience for the player. Inspired by the history of the genre, the game has a minimalistic pixel-art framework with simple yet strategic weapons, power-ups and abilities. Every level is completely procedurally generated, including the enemy sprites, behaviour, and wave patterns. The environmental features are also dynamically created using a combination of Poisson disc sampling, fractal noise curves, Delaunay triangulation, and Voronoi partitioning.

Declaration

I declare that the material submitted for assessment is my own work except where credit is explicitly given to others by citation or acknowledgement. This work was performed during the current academic year except where otherwise stated.

The main text of this project report is 11844 words long, including project specification and plan.

In submitting this project report to the University of St Andrews, I give permission for it to be made available for use in accordance with the regulations of the University Library. I also give permission for the title and abstract to be published and for copies of the report to be made and supplied at cost to any bona fide library or research worker, and to be made available on the World Wide Web. I retain the copyright in this work.

Contents

1 Introduction **1**

2 Context Survey **2**
2.1 Shoot 'Em Ups . . . 2
2.1.1 Definition . . . 2
2.1.2 History . . . 3
2.1.3 Game Design . . . 6
2.2 Procedural Generation . . . 8
2.2.1 History . . . 8
2.2.2 Techniques & Applications . . . 9
2.2.3 Examples in Shoot 'Em Ups . . . 12

3 Requirements Specification **13**
3.1 Primary Requirements . . . 13
3.2 Secondary Requirements . . . 13

4 Software Engineering Process **14**
4.1 Development Methodology . . . 14
4.2 Tools & Technologies . . . 14
4.3 Testing . . . 15

5 Ethics **16**

6 Design **17**
6.1 Overview . . . 17
6.2 Visuals & Audio . . . 18
6.3 Player . . . 19
6.3.1 Movement . . . 19
6.3.2 Weapons . . . 19
6.3.3 Powers & Shop . . . 20
6.3.4 Scoring . . . 21
6.4 Enemies . . . 21
6.4.1 Sprites . . . 21
6.4.2 Behaviour . . . 22
6.5 Levels . . . 23
6.5.1 Waves . . . 23
6.5.2 Environment . . . 23

7 Implementation **24**
7.1 Core Game . . . 24
7.1.1 Management . . . 24
7.1.2 Visuals & Audio . . . 24
7.1.3 Menus & UI . . . 25
7.1.4 Controls . . . 26
7.1.5 Weapons & Powers . . . 26
7.1.6 Projectiles & Collision . . . 27
7.2 Level Generation . . . 27
7.2.1 Sprite Parameters . . . 28
7.2.2 Sprite Textures . . . 29
7.2.3 Enemy Behaviour . . . 30
7.2.4 Level Structure . . . 31
7.3 Environment Generation . . . 31
7.3.1 Poisson Disc Sampling . . . 33
7.3.2 Fractal Noise Curve . . . 33
7.3.3 Delaunay Triangulation . . . 34
7.3.4 Voronoi Diagram . . . 34
7.3.5 Texture Loading . . . 35

8 Evaluation & Critical Appraisal **36**
8.1 User Testing . . . 36
8.2 Requirement Fulfilment . . . 38
8.2.1 Primary Requirements . . . 38
8.2.2 Secondary Requirements . . . 39

9 Conclusion **41**

References **41**

Game Bibliography **45**

A Ethics Approval Form **47**

B Player Guide **48**

C Playtest Feedback **49**

1 Introduction

Shoot 'em ups, also known as shmups, are a genre of action video games with simple yet intense gameplay originating from the arcade era. These games typically feature ranged combat between a protagonist and multiple antagonists, as well as a scrolling top-down or side-view perspective [1]. From their establishment with the hit game *Space Invaders* (1978), shoot 'em ups were popular throughout the 1980s with iconic titles such as *Gradius* (1985) and *R-Type* (1987). Since then, the genre has become increasingly niche but has a passionate enthusiast following.

Despite the limited volume of handcrafted content in many shoot 'em up games, there is a strong emphasis on replayability. Mastery is encouraged through difficult challenges, scoring systems and looping levels. Many popular games in other genres, from *Dwarf Fortress* (2006) to *Minecraft* (2009), use procedural content generation to promote variety and replayability while conserving development resources. Recently, this has seen a resurgence as a main element of popular roguelike games such as *Dead Cells* (2018) [2].

This project focused on developing a shoot 'em up game called *Instar* that uses procedural generation as a core component. The aim was to incorporate algorithmic generation into the core aspects of the game, creating an exciting yet consistently enjoyable experience for the player in each new iteration. In particular, the levels of the game are completely procedurally created, including the wave patterns, enemy sprites and statistics, as well as the surrounding environment. The project is built upon game design lessons from the long history of the genre and established techniques for the procedural generation of levels, visuals and mechanics.

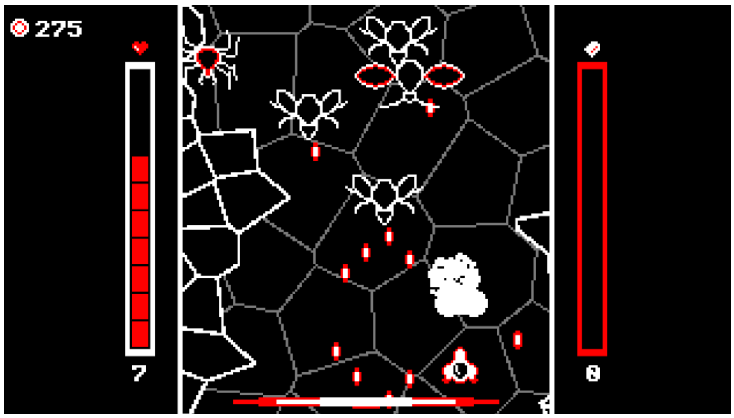


Figure 1: A screenshot of the final gameplay of *Instar*.

2 Context Survey

This section contextualises and discusses the two defining elements of the project, namely shoot 'em up games and procedural content generation, which will inform the game design and implementation. An overview is provided of the history and existing literature in both fields, alongside explanations of popular elements and techniques used in games.

2.1 Shoot 'Em Ups

Designing any game within a genre requires an understanding of its complex history and conventions, in order to innovate on these and appeal to a well-versed audience. The first step is the contentious question of defining the genre.

2.1.1 Definition

The term *shoot 'em up* originates from the arcade era, where it referred to a specific set of games with a highly similar concept: a spaceship with horizontal and/or vertical movement shooting down hordes of enemy craft. This was often shortened to *shmup*, or, given the limited variety of games at the time, simply *shooter*. With the broadening range of game mechanics in the following decades, these terms have become increasingly nebulous and classification demands further distinction. This has become a frequent subject of debate within fan communities [1].

Common elements used to define shoot 'em ups are: a 2D top-down or side-view perspective with a game space that is fixed or scrolls along a single axis; a singular protagonist with ranged weaponry, often a vehicle with a fixed orientation; and multiple enemies that require fast reaction time or pattern memorisation to defeat [1, 3, 4]. Some opt to place restrictions on what elements may not be included, such as a physics system [1]. Games in the genre can be classified into several subgenres, although those in the latter part of the list may be treated as separate genres instead [3]:

- *fixed shooters* restrict the game space to a single screen, with the player only moving along a single axis (see Figure 2);
- *scrolling shooters* are most synonymous with the genre, with the play space scrolling at a fixed speed horizontally, vertically or even isometrically (see Figure 3);

- *multidirectional shooters* or *twin-stick shooters* allow shooting in any direction around the player (see Figure 4);
- *rail shooters* involve the player moving along a fixed route, only able to shoot and dodge (see Figure 5); and
- *run and gun* games feature multidirectional or platform-based movement (see Figure 6).

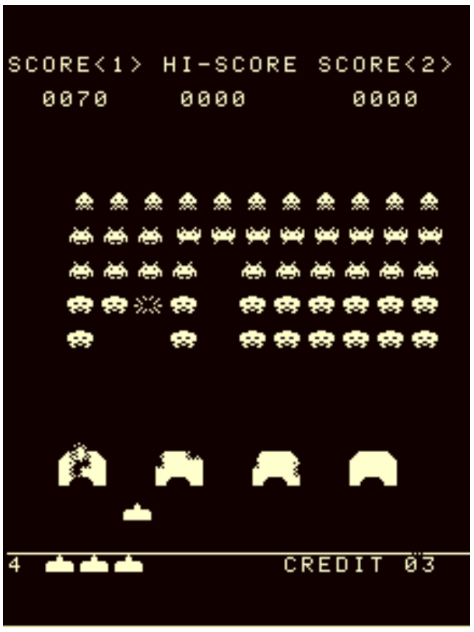


Figure 2: *Space Invaders* (1978).

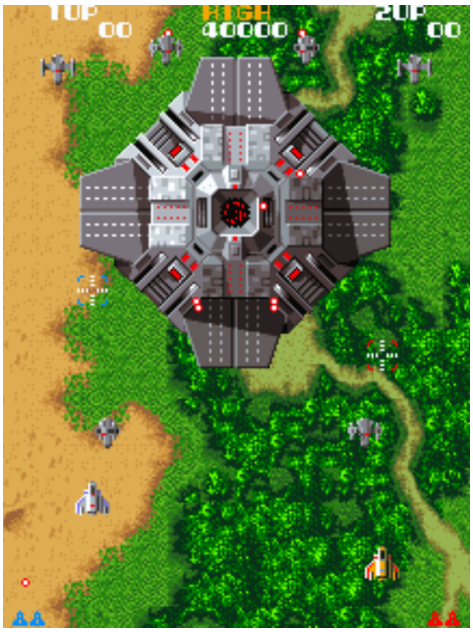


Figure 3: *Xevious* (1982)

As expected, there are several examples of games that defy some of these conventions but may still be regarded as part of the shoot 'em up genre. These are unpopular singular ideas or new trends pushing the boundaries of the style. Nevertheless, in many of these cases, a meaningful relationship to the genre can be established without a categorical judgement.

2.1.2 History

The first video game with shooting mechanics is considered to be *Spacewar!* (1962), an early mainframe game played among its developers at the Massachusetts Institute of Technology that would go on to inspire many arcade video games [5]. With interest in the theme surging in the aftermath of the

space race and *Star Wars* a year prior, *Space Invaders* (1978) was among the first commercial arcade games released. In the game, players control a horizontally moving turret to shoot down alien aircraft, which descend with increasing speed and fire back at the player (see Figure 2). It was a huge commercial success and revolutionary cultural phenomenon, spawning countless imitations that would mark the start of shoot 'em ups [1]. The game also pioneered many genre staples, including high scores, an endless level loop, and a shot limit.



Figure 4: *Robotron: 2084* (1982).



Figure 5: *Star Fox* (1993)

The following decade saw what many call the golden age of shoot 'em ups [2], with a flood of new titles developing the ideas of *Space Invaders*. Games such as *Galaxian* (1979) and *Galaga* (1981) introduced colourful, individualised enemies with distinct movement patterns. Meanwhile, *Gorf* (1981) was among the first to use joysticks for vertical movement in addition to horizontal. Side-scrolling shooters also became popular during this period with games like *Defender* (1981) and *Scramble* (1981). While previous games had limited variety and complexity in their challenges, these newer titles had multiple levels, weapon types and goals (such as saving humans or maintaining fuel) [2]. At the same time, publishers experimented with different formats, for example in *Robotron: 2084* (1982), which popularized twin-stick shooters (see Figure 4), and *Zaxxon* (1982), which had a unique isometric perspective.

As the decade progressed, games became increasingly refined and complex in both their visuals and mechanics. *Xevious* (1982) was a landmark game that became the template for vertically scrolling shoot 'em ups, with highly detailed backgrounds and an introduction of both ground and air targets [5] (see Figure 3). Further iconic games set the standard for new releases. *Gradius* (1985) had distinct, memorisable levels and a strategic power meter system. Meanwhile, *R-Type* (1987) provided an addictive, highly challenging experience with further improved graphics and unique mechanics such as its detachable weapon module (see Figure 7). This period also saw the branching of the genre into subgenres including 'cute 'em ups' like *Fantasy Zone* (1986), as well as wholly new gameplay styles including run and gun games like *Commando* (1985) [2] (see Figure 6).



Figure 6: *Commando* (1985).



Figure 7: *R-Type* (1987).

In the 1990s, a new subgenre known as 'bullet hell' games evolved. First popularised in Japan with the release of *Batsugun* (1993), these games are characterized by complex, highly dense projectile patterns, often with a protagonist that has a tiny hitbox to weave through them [2] (see Figure 8). Another seminal game in the subgenre was *DonPachi* (1995). Such games marked a point where the genre began increasingly to cater to dedicated fans who craved difficult challenges, while mainstream attention shifted to 3D first-person shooting games [1].

Only a few well-known and critically acclaimed titles released during this time, such as *Radiant Silvergun* (1998) and its sequel *Ikaruga* (2001) with its praised polarity mechanic (see Figure 9).

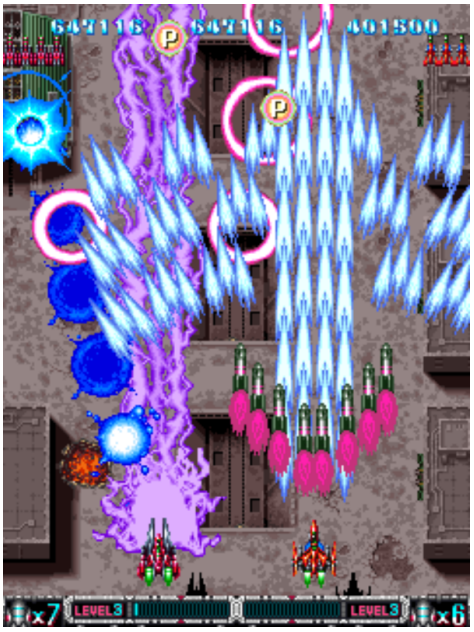


Figure 8: *Batsugun (1993).*



Figure 9: *Ikaruga (2001).*

Today, the genre mostly has a niche appeal to enthusiasts. Games often appear on subscription services such as Xbox Live Arcade, or on the mobile platform [1]. Many shoot 'em ups are indie or *doujin* games developed by small teams, such as the long-running *Touhou* series (1997-).

2.1.3 Game Design

The trends in the game design of shoot 'em ups have increasingly solidified over their lifespan, with many enthusiasts having strong opinions on the subtler elements of the genre. It is crucial to understand these in order to cater to both hardcore fans as well as more casual players.

The primary element of any game in the genre is the player avatar and its controls. It is vital that these are responsive and precise due to the fast-paced nature of shoot 'em ups. In particular, games like *Jets'n'Guns* (2004) with inertia in their movement are frequently classified in the derogatory category of 'euroshmup' [3]. Clear visual design and differentiation of player and enemy bullets further helps players manage game situations. Additionally,

player weapons and power-ups must be carefully balanced between unsatisfying, weak attacks and mindless destruction that removes the emphasis on dodging [6]. The player's effectiveness must also scale with the challenges of levels, as games like *Gradius* (1986) quickly become frustrating when losing power-ups makes progress nigh impossible.

The second most significant component is the enemy and level design. A variety of distinct enemy roles produce engaging scenarios where players must adapt and prioritize targets. Recognizable behaviour is essential to help players familiarise themselves with challenges. However, the main threat should arise from wave patterns rather than individual enemies, which may become frustrating if their health is too high [7]. A good rhythm of enemy waves keeps the action intense while allowing brief moments of respite as contrast. Meanwhile, clever placement of waves and environmental features also creates interesting positional and timing problems for the player, forcing them to plan ahead [6]. Bullet patterns further encourage precise movement. Static bullets, for example, may block certain zones on the screen, while aimed bullets prevent the player from standing still [3]. Complex, layered patterns create intricate challenges, as players must master macro-dodging groups of bullets and micro-dodging in the spaces between individual bullets.

The difficulty of these challenges is an indispensable element of shoot 'em up games. Staying close to their arcade origins, popular games from *R-Type* (1987) to bullet hells provide a high level of difficulty that encourage mastery. Although many provide some tolerance for mistakes, such as continues or extra lives, players often pride themselves on *1CC* (one credit clear), *2-ALL* (twice in a row) or *TLB* (true last boss) achievements [3]. The level of challenge in these games if often a core part of the experience, and makes it necessary for players to spend time and truly engage with the mechanics. More recent western games like *Jamestown* (2011), however, have adopted difficulty options as a way to widen the target audience and make games more accessible, while still encouraging players to push their personal limits.

Difficulty contributes greatly to the replayability of games of the genre. Despite their lack of large volumes of content such as story or customisation, many fans invest significant time learning every detail of a game. The goal is to minimize mistakes or optimize high score. Therefore, many games feature unique and interesting scoring systems that promote different gameplay styles [7]: kill chaining within a short period in *DoDonPachi* (1997); proximity to enemies in *Ketsui* (2003); bullet proximity in *Psyvariar* (2000); or bullet cancelling after letting bullets build up in *Mushihimesama* (2004). Scoring and difficulty may interact in interesting ways through mechanics like rank, where the challenge increases with greater performance, making players manage the risk and reward of a high score [3].

2.2 Procedural Generation

Procedural generation in video games involves the algorithmic creation of content such as levels, characters and items, with limited to no direct input from the player or designer [8]. This subset of generative methods has the unique challenge of forming content that is immediately playable, adhering to the rules and goals of its game. It has become a central tool in the industry for its cost-effective implementation of variety in service of player retention.

2.2.1 History

The first instance of procedural generation in games was seen in analogue roleplaying games like *Dungeons & Dragons*, where lookup tables and dice rolling could aid game runners in generating ideas for encounters or dungeons [9]. Its introduction to digital games emerged in the 1980s as a necessity due to the limited storage capabilities of contemporary devices [8]. *Elite* (1984) overcame the problem of saving game data by using seed numbers to recreate its expansive galaxies at runtime. Another classic procedurally generated game is *Rogue* (1980), whose random dungeons for every new game went on to inspire an entire genre of 'roguelikes' [10] (see Figure 10). However, these games had their own disadvantages, notably their lack of visual appeal.

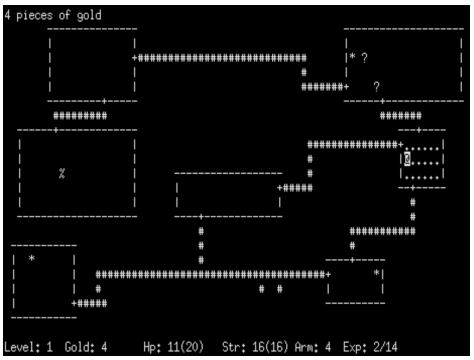


Figure 10: *Rogue* (1980).



Figure 11: *No Man's Sky (2016).*

With the improvement of storage technologies, fewer procedurally generated games were developed until the 1990s, where popular games began to realize its potential. The iconic turn-based strategy series *Civilization* (1991-) originated from this period, as well as the *Diablo* series (1997-), which was among the first to modernize earlier concepts using procedural dungeons with random layouts, loot and monsters. Meanwhile, more sophisticated algorithms for terrain generation were adopted in RPGs like *The Elder Scrolls*

II: Daggerfall (1996), and more individual assets such as trees were created using procedural methods [10]. The mainstream popularity of procedural generation surged in the 2000s with the release of *Minecraft* (2009), the best-selling game of all time, that was inspired by the ASCII world generator *Dwarf Fortress* (2006). Its success legitimized these techniques as a way to create vast, sandbox worlds for players to explore.

Procedural generation has become increasingly diversified, with many developers experimenting with new applications. For instance, *Left 4 Dead 2* (2009) alters game parameters dynamically to improve pacing and adjust difficulty, while *Shadow of Mordor* (2014) uses a 'nemesis system' to respond to player actions in the story [8]. The roguelike genre and hybrids have also had a resurgence in popularity with games such as *Spelunky* (2008) and *The Binding of Isaac* (2011). Procedural generation is increasingly ambitious, with the universe of *No Man's Sky* (2016) containing more than 18 quintillion generated planets (see Figure 11). Even in games not built around them, they are omnipresent in assets, terrains and textures [10].

In the future, procedural generation promises to become increasingly responsive and diverse with the integration of machine learning and AI to allow for more personalized content. However, hybrid approaches will likely remain essential, as algorithmic content has yet to reach the nuance and creativity of handcrafted elements. Research into computationally generated rulesets and game ideas is still in its infancy [11].

2.2.2 Techniques & Applications

The following are common types of procedural generation that have emerged, each specifically implemented and combined to suit the needs of the game.

Modular

The most simple yet most versatile procedural technique is the combination of predefined or dynamically created modules, striking a balance between handcrafted and random generation [8]. A variety of content sections are created, such as chambers, props, and weapon abilities, which can then be procedurally connected and integrated into the game. This is often governed by logical rules like the correct alignment of edges in tiles. In this manner, developers can ensure the design integrity of important components, such as fair and interesting room layout in *Spelunky* (2008), while providing a new experience each time, such as with different combinations of rooms, enemies and treasure.

Fractal and noise-based

Noise-based generation is frequently used to create terrain and similar textures. A landscape can be represented as a heightmap, a matrix of the height of the terrain at each point, or a voxel grid, which divides the three-dimensional space into cubes as in *Minecraft* (2009). A noise function generates smooth, continuous random values over such a space that mimics natural, rough terrain [12]. Typically, functions define a lattice of random values (e.g. value noise) or gradients (e.g. Perlin or simplex noise), and use interpolation to calculate the points in between in a manner that smoothly blends between lattice values.

The above produces a terrain that changes at a constant frequency. For more organic terrain, incorporating larger oscillations like mountains as well as smaller ones like crags, multiple octaves of noise with different frequencies and amplitudes can be added together. This self-similar process is called fractal terrain (see Figure 12). This may be implemented directly for a small number of octaves, or make use of advanced approximation algorithms – such as the diamond-square process – for complex formations with fine details [8].

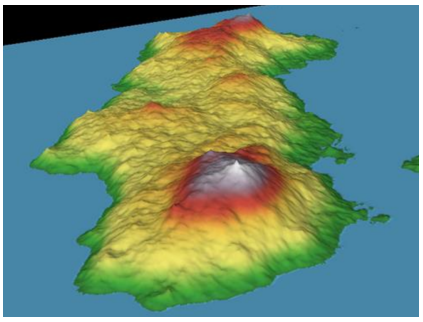


Figure 12: Fractal terrain generated using a noise function.



Figure 13: Trees generated using an L-system.

Grammar-based

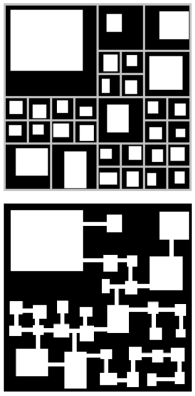
Grammars are a simple yet powerful tool for creating self-similar and rule-based content. A (formal) grammar is a list of production rules for recursively rewriting strings by replacing symbols [13]. Lindenmayer systems (L-systems) are a type of grammar commonly used in games. In this system, rules are applied to the string in parallel, rewriting all possible symbols in the same iteration. This can be used to generate fractal shapes that are commonly found in vegetation, landscapes and cities [12]. For example, *SpeedTree* is a popular middleware in AAA games such as *No Man's Sky* (2016) that

generates trees and foliage using L-systems (see Figure 13). Other grammars may be used to create logical missions with multiple tasks leading up to a goal [8].

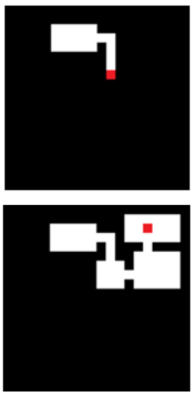
Spatial

A particularly common problem in procedural games is the generation of dungeons, caves and similar play spaces. A variety of techniques exist for this purpose with varying complexity. First among these is space partitioning, usually binary space partitioning, which involves recursively splitting a space horizontally or vertically into two disjoint cells [8] (see Figure 14a). The resulting set of randomly sized rectangles can each contain a chamber without risk of overlapping, after which connecting hallways may be drawn.

In contrast to this macro approach, agent-based methods follow a micro approach where the algorithm has no knowledge of the entire play space. Using this technique, an agent moves around the space to 'dig' hallways and rooms (see Figure 14b). This often leads to less structured, organic levels. However, due to this chaotic nature, such an agent requires complex, well-tested behaviour to avoid issues with playability [8].



(a) Binary space partitioning.



(b) Agent-based digging.



(c) Cellular automata.

Figure 14: Various methods to generate game spaces.

Another method with similar problems is cellular automata. This concept relies on a grid where each cell can be in one of several states. A transition function defines a way for these states to evolve based on surrounding cells [13]. This creates a very natural, eroded look based on relatively few parameters (see Figure 14c). Once again, significant care must be taken to ensure correctness, as designers have little direct control over what gets generated.

Search-based

The search-based approach to procedural generation is currently mostly investigated in academia due to its complexity. In search-based generation, a given search space is analysed by an optimisation algorithm to find a satisfactory solution [14]. There are three important components to this method.

Firstly, the required content needs a representation that defines what can be generated. In *Infinite Mario Bros.*, researchers experimented with more direct representations of levels, as a list of positions of blocks, and more indirect ones, as a list of desirable properties [8]. The former is highly expressive, while the latter has far fewer dimensions and is thus easier to search. This method is therefore difficult to adapt to highly complex games.

These representations are evaluated by an evaluation function, which is a measure of the playability or other desirable aspects of the content. Such a function can be very challenging to define due to the subjective nature of play experience [14].

Lastly, a search algorithm uses the evaluation function to search the representation space for a solution with a high evaluation score. This is usually an evolutionary algorithm, which produces generations of individuals and selects the fittest ones to reproduce via mutation or recombination [13].

2.2.3 Examples in Shoot 'Em Ups

Procedural generation was not yet well-developed at the height of shoot 'em up arcade games, and so classic titles overwhelmingly contain handcrafted levels, enemies and weapons. Occasionally, minor random elements were used to make gameplay less predictable, such as the order of some levels in *Tempest* (1981). However, the effect was usually achieved with hard-coded data tables. A notable outlier is *River Raid* (1982), which used a pseudorandom sequence generated by a linear-feedback shift register to procedurally create its terrain [15]. The game did have a fixed seed, though, so it generated the same level each time. Meanwhile, *Zanac* (1986) had a unique 'Automatic Level of Difficulty Control' system that changed enemy patterns based on the performance of the player.

Many shoot 'em up enthusiasts still prefer predictable games that can be memorized for perfect completion. However, casual games in particular frequently add replayability with endless modes that randomly spawn enemies and hazards until the player eventually dies. Examples include *Geometry Wars: Retro Evolved* (2003) and *Nova Drift* (2024). Moreover, developers have blended shoot 'em ups with the highly popular roguelike genre, such as in *Steredenn: Binary Stars* (2015), which generates completely new levels with different enemy formations for each session.

3 Requirements Specification

The requirements for the project were established in the initial DOER document, and have remained largely unchanged throughout its duration.

3.1 Primary Requirements

The primary requirements are those that are central to the success of the project:

1. Create a playable shoot 'em up framework with essential game mechanics, including a level system, player movement and shooting, as well as enemy and environment interaction.
2. Develop a procedural generation algorithm for levels populated with environmental features and waves of enemies.
3. Develop a procedural generation algorithm for the visuals, movement and attacks of enemies.
4. Test the game to ensure generated content is fair and fun for players.

3.2 Secondary Requirements

The secondary requirements are additional objectives that are achieved if possible without detracting from the primary ones:

1. Develop a procedural generation algorithm for player weapons and power-ups.
2. Develop an interesting scoring system that rewards and encourages players.
3. Extend the level design algorithm to incorporate improved pacing and difficulty progression.
4. Extend the enemy design algorithm to incorporate unique abilities and bosses.
5. Polish the game visuals and mechanics to improve the game feel, clarity and aesthetics.

4 Software Engineering Process

The approaches and tools used in the development of the game are covered in this section. The effective management of the project was essential to handle the complexity of the many parts of the game and meet deadlines.

4.1 Development Methodology

The software development methodology chosen for the project followed the Agile model, with an emphasis on flexibility and iterative improvement. As opposed to the strict plans and timelines involved in a traditional cascade approach, development happened in shorter sprints. The weekly meetings with my supervisor served as an opportunity to reflect on progress, discuss any issues that arose, and define goals for the next week. However, no formal framework such as Scrum was adopted, as the overhead these introduce are typically more advantageous for larger projects and teams.

The rapid prototyping and testing of ideas was particularly important in a video game context, where game feel and fun are notoriously difficult to evaluate without experiencing them. Procedural generation algorithms may similarly prove to be unpredictable in their results. As the game evolved, the Agile method allowed me to adapt to particularly interesting creative ideas and discard or rework unsuccessful ones. It also improved the debugging process, as I could test and fix systems in isolation before moving onto the next. Furthermore, it suited the changing workload I had from week to week with other university projects. One disadvantage the approach introduced was that systems were not always built with the needs of future features in mind. However, frequent refactoring for improved extensibility helped mitigate this issue.

4.2 Tools & Technologies

The following technologies and development tools were utilised throughout development.

Unity

The Unity engine is highly popular for games of small to medium scope for its versatility, extensive support and ease of use for newer developers. For a 2D non-commercial game with no specific technical requirements, this was a clear choice, particularly so because I already had relevant experience with the framework. It also provides Unity Version Control, which is less powerful than Git but entirely sufficient for a single developer.

Visual Studio Code

Scripts for Unity are written in the C# language. Visual Studio Code provided an ideal environment for development with many helpful features to speed up programming as well as built-in integration with Unity.

Trello

I maintained a Trello board for oversight of user stories and unresolved bugs in the game. This simple interface was useful in tracking the state of tasks and subtasks, organised according to the Agile sprint goals.

4.3 Testing

For the majority of the development process, testing focused on informal, exploratory tests rather than extensive automated testing. While the latter were useful for complex algorithms and back-end systems, the game itself places more importance on visual feedback, player experience and emergent interactions. These are most effectively evaluated by frequent hands-on playtests, which enabled rapid fine-tuning of balance and responsiveness. Additionally, they helped identify many unexpected edge cases in the interactions between the numerous systems. The testing of the procedural generation systems, meanwhile, was aided by custom Unity editor windows that allowed for generating many procedural instances without running the entire game.

5 Ethics

The only ethical consideration regarding this project is the involvement of human participants in a user evaluation of the artefact. This process was limited to an hour for each participant and did not involve the collection of personal data. Thus, it is covered by the artefact evaluation form, a signed copy of which can be found in Appendix A.

6 Design

In this section, an overview is given of the game and software design decisions that led to the final *Instar* game. These were essential in guiding the implementation and maintaining a focus on the player experience.

6.1 Overview

The project had two initial required components: the shoot 'em up genre and procedural generation. Before considering these in more detail, however, the broader concept of the game was formulated. There are a variety of frameworks for defining the foundational elements of a game, such as Katie Salen and Eric Zimmerman's 'meaningful play' [16] and Jesse Schell's lenses [17], but the core considerations are always centred around player experience.

First, it is important to understand the players themselves that are considered the primary audience. For almost all shoot 'em ups, they fall into two categories according to the Bartle taxonomy of player types [18]: *killers*, who enjoy improving their skill to prove themselves superior to others, and *achievers*, who want to achieve measurements of success such as high scores and game completion. Moreover, based around my own relationship to shoot 'em ups, *Instar* is aimed at players who might not yet have the genre skill and literacy from playing many such games. The intent remains to create tense, exhilarating gameplay that encourages an immersive 'flow state'. Players should be excited to improve their mastery and tackle new procedurally generated challenges.

Every element of the game serves to support and reinforce these aims. Following the MDA framework [19], the designer can only influence the *aesthetics* that the player experiences through *dynamics*, which are the systems and interactions of the game. In *Instar*, the goal is to produce a simple yet satisfying moveset, varied enemies that create interesting, strategic situations, and rhythmic pacing that steadily increases in difficulty. These are in turn created by the *mechanics*, which encompasses the rules, generation, art, etc. that are explained in further detail below.

An important consideration was the extent of the procedural generation, ranging from only supplementary features to encompassing the entire game. As this was a main focus of the project, procedural generation would be a significant component, in particular for the levels including wave patterns, enemies, and environments. Meanwhile, the player-related elements, such as weapons and powers, are not procedurally generated: they require less variety and have a higher standard of quality, making handcrafting more

efficient. Because the game is centred around skill-based action rather than exploration, the aim is to provide a good diversity in the generation but prioritise quality and consistency. Levels should all be playable and have a solid minimum standard, avoiding glaring irregularities even if this somewhat reduces the generation space. Each session is generated from a single seed, so that any configuration may possibly be replayed if it proves to be particularly interesting.

6.2 Visuals & Audio

The game has a highly minimalist aesthetic, with low-resolution pixel art and a sparse colour palette of only three colours (see Figure 15). This gives the game a distinctly retro aesthetic that is reminiscent of the best classic shoot 'em ups, and allows the player to focus on the gameplay with little visual distraction. The imperfect distortions and jittery movement that result from objects snapping to a pixel grid provide an abstract and timeless feel. Furthermore, the low resolution improves the procedural generation aspect both in terms of performance and coding complexity. There is a similar benefit for hand-drawn assets. The monotonicity of the limited colours is alleviated by the option to use one of several different palettes. However, distinguishing each element on the screen with limited colour options was an important consideration throughout.

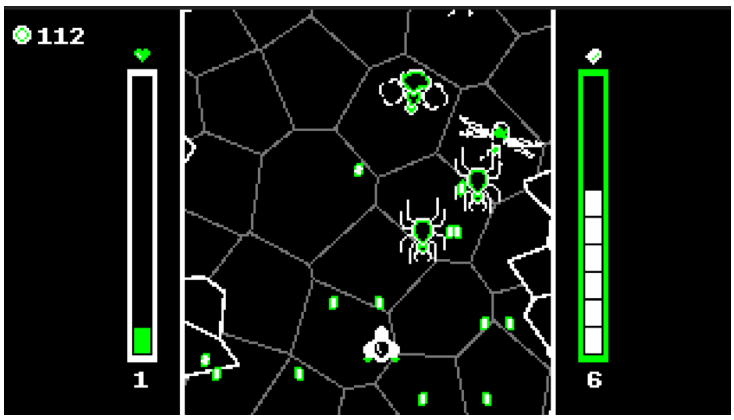


Figure 15: A screenshot showcasing the minimal colour pixel art aesthetic of the game, with a green highlight selected.

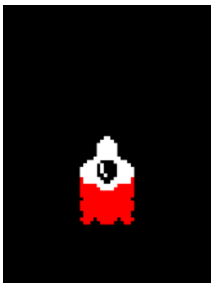
For the audio, customisation would be more time-consuming and detract from the focus of the project. Although this resulted in standard sci-fi sound effects and music, the simple, abstract soundscape fit the theme of the game.

6.3 Player

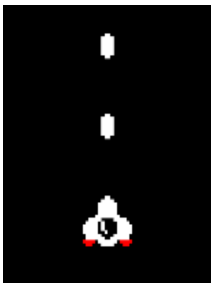
The character controlled by the player is a nameless craft with a limited set of capabilities intended to flow together in an intuitive but strategic manner.

6.3.1 Movement

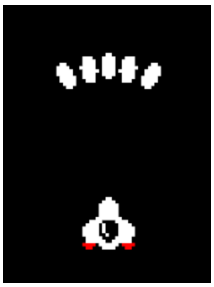
The player can move around the screen in a standard manner for most shoot 'em ups, notably without any inertia to remain immediately responsive to small movements. Beyond this, they have an additional movement ability that allows them to dash in a straight line in the direction they are moving, moving through projectiles unharmed in the process (see Figure 16a). This is a useful tool for moving long distances quickly and dodging attacks, without relying on an overly fast player speed that would compromise precise control. The dash offers some safety to the player at all times, while retaining a strategic depth as they account for where they end up and how to manage its cooldown. Other defensive and mobility abilities were prototyped, such as slowing time and projectile deflection. However, the dash draws less emphasis, fitting the aim of complementing rather than outshining the weapons system.



(a) Dashing.



(b) Regular fire.



(c) Special fire.

Figure 16: The three core player actions.

6.3.2 Weapons

The player has two firing capabilities: a regular attack and a special attack. These are both governed by an ammunition system that holds a limited number of shots shared between them. The regular fire mode shoots simple projectiles at a steady rate (see Figure 16b), but the consumed ammunition encourages the player to aim them deliberately as opposed to holding down the fire button and sweeping over enemies. The special attack, meanwhile, is

a more powerful spread shot that deals more damage and can affect enemies to the sides (see Figure 16c). However, it consumes all remaining ammunition, leaving the player vulnerable until they use a dash to reload. This forms a simple loop: fire until the ammunition is almost depleted, use a special attack to maximise the remaining ammunition, then dash and repeat. This mechanic was chosen over others such as a cooldown or charged attack as it prevents the player from relying on only a part of their abilities – they must use them in conjunction for their full effect. The optimal combination will naturally differ according to the situation as well as the obtained powers.

6.3.3 Powers & Shop

At the end of a level, the player may choose from a selection of three powers that enhance their regular attack (faster fire rate, more projectiles, longer range, etc.), dramatically change their special attack (homing projectiles, lasers, explosions, etc.) or provide another passive benefit (increased damage when hurt, regain ammunition when defeating an enemy, etc.) to the player (see Figure 17). This progression matches the increase in difficulty over levels, and aims to provide an interesting strategic choice for the player. In addition, it serves to differentiate the experience over multiple attempts and provides a sense of excitement and discovery when acquiring new synergies.



Figure 17: The powers screen.

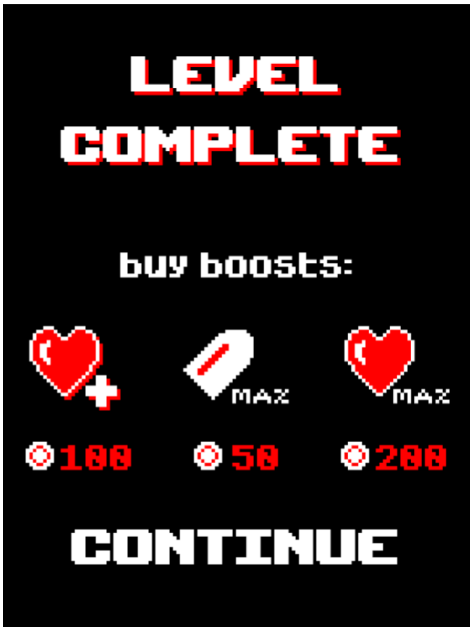


Figure 18: The shop screen.

After selecting a power, the player may also purchase basic improvements with the score they acquired: healing, increased maximum health, and increased ammo capacity (see Figure 18). Their presence after every level ensures consistency in the expected basic capabilities of the player as the difficulty progresses. It also organically supports the increase in player skill over multiple attempts. At first, players may buy healing after each level to stay alive until they meet an obstacle they cannot overcome. As their mastery improves, they will not take as much damage and have more currency to spend on future-oriented upgrades, allowing them to progress to later levels.

6.3.4 Scoring

Score serves as both an end target to strive for and a useful currency to collect during levels. The total score is the high score displayed at the end, but it can also be spent at the shop for crucial healing and upgrades. Therefore, score is valuable to experienced players, who want to show off their achievements, and to inexperienced players, who only want to survive as long as possible. The scoring system encourages the 'intended' playstyle, so that the most optimal way to play is also the most interesting or fun. In this case, this is consistently defeating enemies before they leave the screen, while avoiding taking damage. Thus, score is awarded for defeating enemies, and can be boosted by a streak system that rewards doing so many times in a row without being hit yourself.

6.4 Enemies

A variety of enemies spawn in levels to challenge the player. This is one of the main procedurally generated components of the game.

6.4.1 Sprites

The appearance of each enemy must be simple, recognisable, and easily differentiated. For this purpose, the theme of insects was chosen. Insects often have fairly basic shapes, making them ideal for pixel art, and are relatively homogeneous while having numerous distinguishing features such as horns, antennae, and wing patterns (see Figure 19). Furthermore, their popular dislike have made them an iconic enemy in many video games, though not as typical for shoot 'em ups as aliens and planes. The title of the game, *Instar*, refers to the cycles of transformation these insects undergo during their development. The vertical layout was also chosen to complement the insect sprites, which are more interesting from above than from the side.

The generation process itself involves modular parts for the body, legs, wings, etc. This ensures consistency, as each sprite is constructed from a verified combination of components. Each part, however, can take one of several different forms, and additional random variation is applied to the shapes and positions to improve diversity. To give players a better insight into the capabilities of an enemy at first glance, more dangerous enemies have more coloured highlights.

6.4.2 Behaviour

The capabilities and behaviour of the enemies are also procedurally generated. These include their statistics such as health and speed, their weapons statistics such as fire rate and spread, as well as special abilities like lasers and explosions. The player must observe and adapt to the new enemies during every session, preventing them from resorting to repeating the same tactics. Each enemy is generated with a particular intended difficulty, which is allocated between strengths and weaknesses. Balance is a crucial concern here, as ideally groups of the same difficulty will be equally challenging to the player. However, this must be weighed against providing a broad space for generation for more diversity.

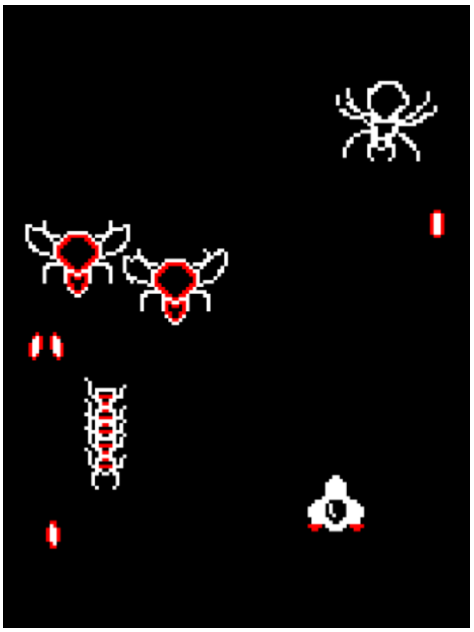


Figure 19: A group of different enemies attacking the player.

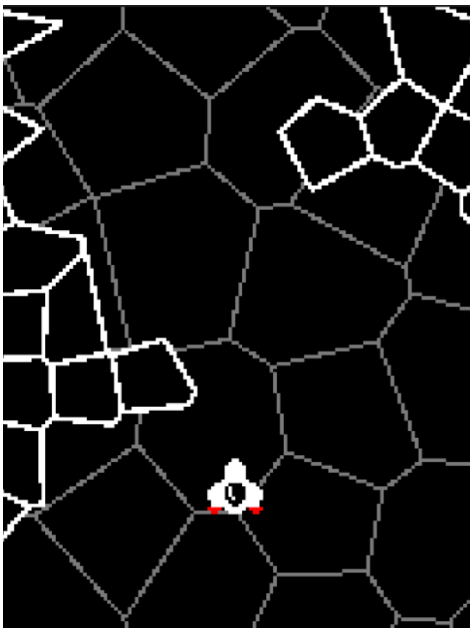


Figure 20: The curves and protrusions of the cell-like environment.

6.5 Levels

The structure of the game follows a typical arcade game, where players move through a series of levels until their lives run out and they must restart completely. This fits the procedural generation, as new levels are created every attempt as opposed to a pre-made set. Due to procedural generation, it is possible to have endless non-repeating levels. Although a fixed ending would be appealing to some players, the endless structure allows for continuous competition. The development resources were ultimately better focused on the parts of the game that the most time is spent on.

6.5.1 Waves

The sequences of enemy spawns are randomly generated to provide a rising difficulty curve with varied pacing for sufficient breathing space between intense sections. Therefore, groups of enemies are spawned in short bursts, with a longer interval afterwards. The initial sections of a level include less threatening enemies that can be quickly defeated, but more challenging ones appear increasingly frequently over time. The end of the level has the most difficult enemies in fast succession. This is meant to raise the level of excitement to an exhilarating crescendo before the pause in between levels.

6.5.2 Environment

The environment is similarly inspired by insects. The background and foreground features are formed using the spatial partitioning technique of Voronoi diagrams, which are a mathematical way of dividing a space to mimic an abstract cell-like structure (see Figure 20). Thus, the player can be imagined to be flying through an insect-built hive. The Voronoi diagrams are generated by first finding evenly spaced points using blue noise, which are then transformed by a Perlin noise fractal curve to create irregular protrusions that force the player to manoeuvre around them. This results in interesting gameplay situations in addition to visual appeal. Delaunay triangulation and Voronoi partitioning are performed on these points, then converted to pixel textures.

7 Implementation

This section details the structure and implementation of the final game in Unity, with an emphasis on the procedural elements. The process involved both scripting in C# and configuring the objects and components in the Unity editor.

7.1 Core Game

The core game systems involve the non-procedural parts of the game, and form a modular framework for inserting those dynamically created components. The game consists of four total scenes: one for running the levels, and three for the start, continue, and end menus.

7.1.1 Management

The central management of the game is handled by a number of singleton classes, which persist their state between scenes. The `SceneManager` is responsible for loading and unloading scenes. Meanwhile, the `GameManager` holds all global data pertinent to the current session, including health, score, ammunition, and powers. As any modification to these occurs in a central class, the states of these variables can always be controlled. In addition, it holds a set of global constants, ensuring these may be easily refactored from a central location. This also allows static classes to access Unity assets, which otherwise can only be referenced by attaching them to a component. Lastly, the `CameraManager` and `MusicManager` classes contribute to consistent visual and audio behaviour, respectively.

7.1.2 Visuals & Audio

The pixel-perfect look of the game is achieved using a static screen resolution of 320×180 and the `PixelPerfectCamera` component. This automatically snaps the game visuals to a pixel grid, converting sprites such as a high-resolution circle to a low-resolution approximation. It is particularly useful for handling the rotation of enemies without specifying separate sprites for the different states. However, this approach also necessitated great care with each sprite and UI element on the screen, as any deviation from the correct pixel size and position would cause the element to appear blurry and imperfect. Therefore, every single sprite and font atlas was created with the exact dimensions in mind, as it could not be resized. For instance, the health and ammunition displays required custom behaviour to calculate a

pixel-perfect segmented bar, as the automatic mapping of a higher resolution bar could cause segments to blend together.

When changing the colour palette, rather than switching out the colours of each element individually, a universal solution is applied with a custom post-processing shader. The `ColorMapShader` processes each individual pixel, calculating the closest Euclidean distance to white, black, red and green. The latter represents the grey colour used in the background details, which would otherwise be difficult to distinguish from white or black. The pixel can then be recoloured to the matching colour in the new palette, with grey always mapping to a desaturated version of colour replacing white. The shader is applied to the entire game image during `OnRenderImage` with a blit operation.

Many of the visual effects in the game rely on the C# event system. These provide a decoupled way of responding to actions in the game such as the player firing, dashing or receiving damage. The logic for these actions simply defines and invokes a single event, and any accompanying visuals subscribe to the event so they are triggered every time the event happens. Some of the visuals use built-in Unity systems, including the `Animator` for player animations, `ParticleSystem` for explosions, and `TrailRenderer` for the ship thrusters. Others run a coroutine script, which may be paused and resumed for time-based operations. These are not truly asynchronous: coroutines align with the Unity update system by executing code on subsequent frames in the main thread. For example, `Flash` switches to a solid white material and back at set intervals, while `ShakeScreen` repeatedly adjusts the camera position by a random vector.

Because of the lack of proximity issues, audio is handled in a very straightforward manner. For each clip, an `AudioSource` is created at the exact position of the `AudioListener`, so that all sounds are played at the same volume. A random clip is played from a short list, to avoid repetitive sounds, and the object is then immediately destroyed. This separation ensures that the sounds are not affected by the original object that triggered them, such as the destruction of an enemy before it can finish playing its death sound effect.

7.1.3 Menus & UI

The menus have simple behaviour that allows the player to navigate them using the arrow keys, which is handled by the `Navigation` component. This defines a generic framework for adding buttons in rows and columns. With each player input, the currently selected component is calculated to intuitively account for a different number of buttons in each row and column.

The buttons themselves all have a component derived from `Interactable` that triggers their required behaviour, such as continuing to the next scene or closing the game.

The UI is implemented with Unity's `Canvas` system, which is compatible with the pixel-perfect camera. Similarly to the visual effects, it heavily relies on events invoked by the game logic. This significantly improves performance, as elements are only updated when relevant values have changed rather than checking for updates every frame.

7.1.4 Controls

Player input is handled through the `InputAction` system, which holds action maps for menu actions (navigation and interacting with a button) and game actions (movement, dashing and using attacks). This is a useful system for binding multiple buttons on different devices and performing basic input processing. For instance, movement direction is retrieved as a normalised vector, and buttons must be released before they are pressed again. The `PlayerInput` coordinates the responses to these inputs and sends them to the relevant child objects. In particular, there is a unique script for firing the main weapon that allows either pressing or holding down the button. For movement, none of the physics components are used, instead modifying the position of the player directly based on speed and frame rate.

7.1.5 Weapons & Powers

Both player and enemy weapons are implemented in a modular manner using Unity scriptable objects. This is a class that can hold data independently from game objects and is stored as a serialisable asset. Thus, weapons can be created and changed independently, then assigned to a `Weapon` component to drive behaviour. Each weapon has numerous attributes, including fire rate, projectile count and burst fire. It also has a scriptable subobject for the projectile, which holds data for damage, speed, range, and special abilities like homing strength.

When the weapon is fired, the `Weapon` component uses this information to calculate the position and rotation of each projectile. The former is given by the owner's fire point, while the latter is calculated using quaternions to form an even spread and randomly modified to account for accuracy. Projectiles are then created at these points, multiple times if the weapon fired in bursts. Afterwards, the weapon is blocked from firing until a cooldown coroutine runs out. Player and enemy weapons are separate subclasses, as they differ slightly in their behaviour. For example, player weapons consume ammunition and

produce a small kickback for additional responsiveness.

Powers function in a similar manner. Each power inherits from a scriptable object with fields including its sprite and description. The class also contains several empty methods called at different parts of the game, such as `PlayerFirePower()`. These can then be overridden in subclasses to produce behaviour such as increasing the fire rate or reducing the range of the main player weapon. Since these scriptable object subclasses produce a single unique behaviour, they each only have one asset instance.

7.1.6 Projectiles & Collision

As projectiles may be procedurally generated, they are individually created by the `ProjectileCreator` from their scriptable object data rather than instantiating a prefab asset. Thus, a new game object is generated programmatically with child objects for the visuals and hitbox. By default, a projectile simply travels forward in the direction it is facing at a set speed, deleting itself when it has travelled the maximum range. However, there is additional functionality for special projectiles, such as rotating towards a target for homing projectiles or following the weapon for a beam attack.

Hit detection uses the Unity collision system, with the player, enemies and projectiles each having a `CapsuleCollider` component. The size of the colliders always favours the player: enemy hitboxes are larger and easy to hit, while the player hitbox is smaller than the sprite. Thus, only substantial collisions are punished, which often feels more fair to the player. When two such components intersect, the `OnTriggerEnter2D` method is automatically called in the attached scripts. The layers of these colliders can be configured to manage which objects should collide, such as enemy projectiles and the player, but not player projectiles and the player. Each object with a collider has a corresponding script that implements the `IHittable` interface, which enables the two components to exchange information on the source of the collision and damage resulting from it. Piercing and explosive projectiles, for instance, use this to keep a list of damaged entities so that damage is only applied once to each.

7.2 Level Generation

The first main procedurally generated components of the game are the levels, in particular including the waves of enemies that they are comprised of.

7.2.1 Sprite Parameters

A major part of the enemy generation are the sprites. Before an insect texture is generated, the procedural algorithm first generates a set of `InsectParameter` enums. These parameters define the general shape and type of the insect, with the following categories:

- head appendages (`NoHeadAppendage`, `Horns`, `Antennae`);
- body shape (`FlatBody`, `ThinBody`, `RoundBody`);
- number of body parts (`TwoParts`, `ThreeParts`, `ManyParts`);
- wings (`NoWings`, `Wings`);
  - number of wings (`TwoWings`, `FourWings`);
  - wing shape (`ThinWings`, `BigWings`, `TriangleWings`);
  - wing position (`SideWings`, `BackWings`);
- legs (`Legs`, `NoLegs`);
  - number of legs (`SixLegs`, `EightLegs`, `ManyLegs`);
  - leg shape (`JoinedLegs`, `CurvedLegs`); and
- highlights (`HornHighlight`, `AntennaHighlight`, `EyesHighlight`, `LegHighlight`, `BodySpotsHighlight`, `WingSpotsHighlight`, `BodyOutlineHighlight`, `WingOutlineHighlight`, `BodyStripesHighlight`).

The enum uses the `Attribute` class to attach a list of exclusions to each parameter, allowing customisation of which combinations are possible. For instance, an insect with many bodies must also have many legs. A set of parameters is chosen by repeatedly taking a random parameter from the remaining ones, then removing the items on its exclusion list as well as any items that have it on their exclusion list. The random selection can be modified using the weight attribute for each parameter, allowing the curation of how common traits are in the population. The total number of highlights can also directly be controlled by a parameter.

Determining the parameters of an insect beforehand is beneficial because it combines the advantages of a modular system with the advantage of a single unified system. Each part of the texture may be generated separately, increasing readability and extensibility, but can also access general information about the rest of the insect to inform its decisions. As an example, if wings are present, shorter legs are generated to reduce visual noise, without requiring these two components to interact.

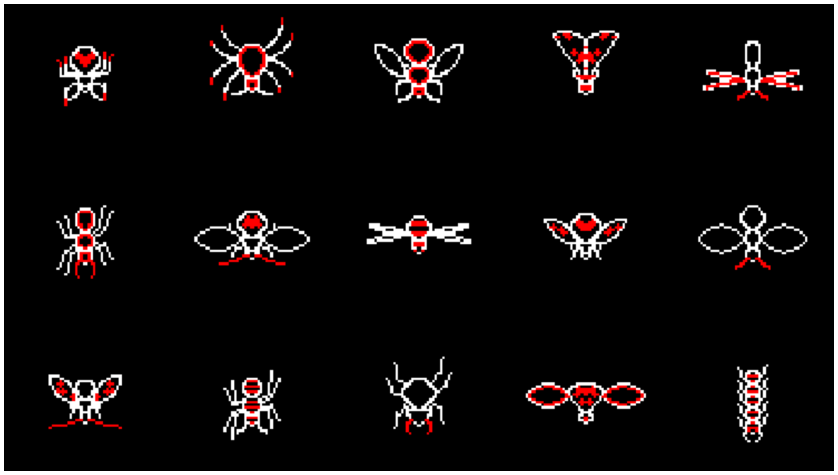


Figure 21: A variety of procedurally generated insect sprites. The parameters of the top left one are: `Legs`, `LegHighlight`, `BodySpotsHighlight`, `NoWings`, `ThreeParts`, `FlatBody`, `JointedLegs`, `SixLegs`, and `Antennae`.

7.2.2 Sprite Textures

After generating the parameters, the texture is created in a 64×64 resolution in the `InsectTextureGenerator` class (see Figure 21 for examples). In this process, an instance of the `Insect` class is generated based on the parameters by iteratively generating its nested `InsectPart` components (`InsectBody`, then `InsectBodyPart`, etc.). It is useful to have a complete object representation of the insect first before drawing, instead of immediately colouring parts as in the first iteration, as the separation of concerns allows for easier modification and debugging of both processes. Furthermore, an insect could be reused with changes to its variables or drawing style.

Once the object representation is generated, the `Color` method can be called in every `InsectPart` to draw it onto a pixel grid. These make use of a number of utility methods to calculate the pixels along a line segment

$$L(t) = (1-t)p_1 + tp_2$$

or quadratic Bézier curve

$$B(t) = (1-t)^2 p_1 + 2(1-t)tp_c + t^2 p_2$$

based on their formulas. In addition, a flood fill algorithm is used to fill in the wings and make the background transparent. Lastly, the highlights can be added on top in a different colour. This involves drawing over certain components, or adding patterns in their interior.

Every insect has a simple two-state animation for their legs or wings. For this purpose, the body part is shifted slightly up, down or to the side in one frame, and shifted in the opposite direction in the other. Most body parts are drawn symmetrically along the vertical midline, so that the amount of data and generation required is halved. For the appendages, however, a non-symmetrical image is created that is then mirrored for the second animation frame.

7.2.3 Enemy Behaviour

The behaviour of each enemy is serialised in a dynamically generated scriptable object (see Figure 22a). Each enemy has a difficulty, which is randomly divided into several categories, including health, tracking capability and weapons. These inform the strengths of the enemy in these categories, with some minor random adjustments. This results in a more diverse set of generated enemies than the alternative method of scaling every attribute by the difficulty.

The difficulty assigned to its weapon is propagated downwards to the method generating its weapon scriptable object (see Figure 22b). In a similar fashion, the difficulty is spread over the weapon attributes, such as fire rate and projectile count, before the remainder is passed to the projectile generator. In addition to speed and size, projectile difficulty has a chance to add special attributes such as laser, explosive, or homing projectiles (see Figure 22c).



(a) The enemy object. (b) The weapon object. (c) The projectile object.

Figure 22: The scriptable objects that drive the behaviour of an enemy.

7.2.4 Level Structure

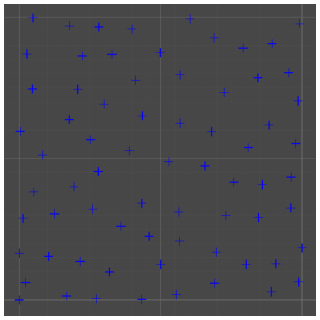
Enemies spawn in waves, which are groups of identical enemies following the same path shortly after each other. Their movement follows a fixed pattern that is randomly selected from a list of several predefined paths. The overall structure of a level is generated to provide varied pacing, with an overall crescendo in intensity at the end. Therefore, waves are generated in short bursts, resulting in challenging groups of enemies before a brief respite. The number of enemies in a wave is inversely proportional to their difficulty, so that easier enemies appear in larger numbers. The final group of enemies is larger, has shorter intervals between waves, and usually contains more difficult enemies.

The levels are run by the `LevelManager`, which is responsible for the coroutines that instantiate `WaveManager` components at the correct intervals as specified by the generation. These in turn create the enemies on the given paths following the data in their scriptable objects. Every session has only two enemies per difficulty, which could appear in three consecutive levels as hard, medium, then easy enemies. This enables players to still observe and learn enemy behaviour, but only within the same session.

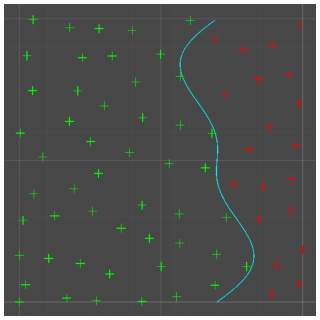
If a player does want to repeat a session, the seed-based generation makes this possible. All procedural components are generated in the same order using a single Unity `Random` seed, resulting in the same levels, enemies and environments each time. Any random effects that may be influenced by the player, such as audio clips and camera shake, use the C# `System.Random` instead. This prevents the course of play from perturbing the sequence of random numbers used in the generation of future levels.

7.3 Environment Generation

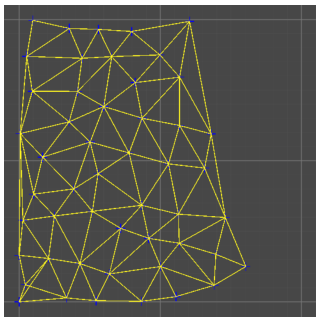
The second major procedural component of the game is the environment generation, with the goal of producing interesting cell-like terrain along the edges of the screen. The process involves generating the cell centrepoints, carving out a terrain curve, forming the cells around the points and then converting them into a pixel texture (see Figure 23).



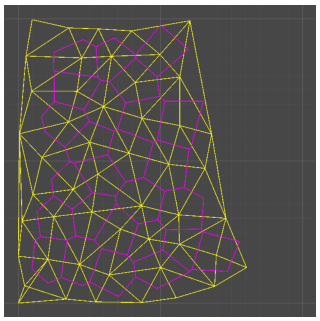
(a) Evenly spaced points are generated using Poisson disc sampling.



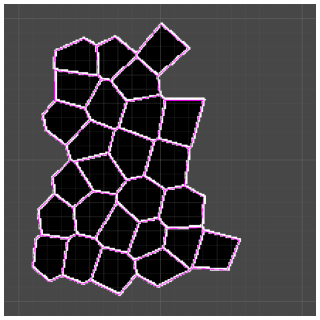
(b) Points are removed following a Perlin noise curve to form terrain.



(c) Delaunay triangulation is performed on the points.



(d) Voronoi cells are calculated using the Delaunay triangles.



(e) The Voronoi cells are asynchronously drawn to a pixel texture.

Figure 23: The procedural generation process of the level environment.

7.3.1 Poisson Disc Sampling

The first step is to create an evenly distributed set of n points within an area that are at least a minimum distance r apart (see Figure 23a). This is necessary as truly random points form cells that are highly irregular in size, which does not match the cell-like goal. Therefore, we can generate the points as blue noise using Poisson disc sampling, which prevents clustering. The method used is a fast $O(n)$ algorithm by Bridson [20], with the following steps:

1. Create an array for storing the samples representing a grid with square sides $r/\sqrt{n}$. This length is chosen such that each square may contain at most one sample. Also create a list for storing active points.

2. Select an initial sample uniformly from the entire area, and insert it into the grid and active list.

3. While the active list is non-empty, choose a random point p from the list. Generate a point uniformly chosen from the annulus around p between radius r and $2r$.

   (a) If the point is adequately far from the other existing points, add it as the next sample to the grid and active list.

   (b) Otherwise, attempt another similarly chosen point. If after a fixed number of attempts, no valid point is found, remove p from the active list.

The grid produced by this algorithm contains the required list of points for the next step.

7.3.2 Fractal Noise Curve

Although the generated points have small irregularities, they do not form varied and interesting terrain within themselves. For this purpose, a portion of the points are carved out by a fractal noise curve to form a more natural shape.

The noise itself is generated using Unity's built-in Perlin noise, which is widely used for controlled, continuous random values in terrain. The frequency and amplitude variables control the scale and intensity of the variation, respectively. Multiple octaves of noise are layered over one another, providing increasingly finer levels of detail. Between octaves, the lacunarity determines the increase in frequency, while the gain dictates the rate that the amplitude decreases.

Once the noise pattern is generated, it is mapped to the size of the area. Furthermore, pillars are inserted with occasional frequency, greatly increasing the height of the curve over a short width. These provide more drastic obstacles to the player. Finally, any points within the curve are retained, while the others are discarded (see Figure 23b).

7.3.3 Delaunay Triangulation

To create Voronoi cells from the resulting points, Delaunay triangulation is first performed. This is an efficient and robust way to find Voronoi cells: due to the dual nature of these structures, one quickly follows from the other. This is a superior method to the initial naive algorithm, which created the pixel texture directly by calculating the distances from every pixel to every point, as it is both faster and provides more useful data for further calculation.

Delaunay triangulation divides the points into triangles whose circumcircles (the circles passing through the three vertices of each triangle) do not contain any points (see Figure 23c). These triangles can be calculated using the Bowyer-Watson algorithm [21] with $O(n^2)$ complexity as follows:

1. Create a list for the triangles and add a super-triangle that contains all points. As the bounding area for the points is known, the straightforward choice is the right triangle containing the rectangle.

2. Iteratively add each point to the triangulation.

   (a) Remove existing triangles in the list with circumcircles containing the point, as these are invalid Delaunay triangles.

   (b) Create new triangles from all unique removed edges by connecting them to the point. Thus, these are valid Delaunay triangles from the perspective of that point.

3. Remove any leftover triangles containing a vertex of the super-triangle.

The resulting triangles are guaranteed to match the Delaunay criteria, and so can directly be converted to Voronoi cells.

7.3.4 Voronoi Diagram

A Voronoi diagram divides the area into cells around the 'site' points, where each point in a cell is closer to its associated site than any other site. The edges are equidistant to two sites; the vertices are equidistant to three. From

the construction of the Delaunay triangulation, the circumcircles do not contain any sites, and the sites on the circumference are equidistant to the centre. Hence, each circumcentre is a Voronoi vertex. This is used to match the Voronoi sites with their vertices to obtain a final list of cells [22]. There are additional checks to ensure cells have no missing edges, are within bounds, and do not have any overly long edges that may result from the irregular noise-based deletions (see Figure 23d).

Since the Voronoi cells have all their data stored as objects, it is possible to calculate their hitbox to detect collisions with the environment. The outer boundaries are calculated filtering all edges that are not shared between cells. These can then be arranged into one or more continuous paths that each form a polygon. Using these polygons, `PolygonCollider` components are constructed, which can then be combined into a single efficient `CompositeCollider`.

7.3.5 Texture Loading

Lastly, the Voronoi cells are converted into pixel grid that is used for a sprite texture (see Figure 23e). This is a straightforward but time-consuming process, as each pixel must be calculated individually. First, the program checks whether a pixel lies on an edge by calculating the distance to the projection point of a normal line. Subsequently, it checks whether a pixel lies within any cell using the outer boundary. This is achieved by casting a horizontal line from the point. If it crosses the boundary an odd number of times, it is inside; if it crosses an even number of times, it is outside.

Because the load time would be extremely high if the textures for an entire level were loaded at once, particularly since three separate textures are required for the background and the two sides, they are calculated in increments. For the initial load, the `LevelCreator` runs the calculation as quickly as possible with each row in parallel. Subsequently, as the player slowly moves through the level, new sections are calculated via an asynchronous, less resource intensive process. These are then combined with the existing texture to extend the level.

8 Evaluation & Critical Appraisal

This section evaluates the final product supported by feedback from a group of playtesters, with respect to the original requirements and in the context of other similar games in the genre.

8.1 User Testing

Player feedback was elicited before finalising the project to inform the last set of balance changes and bug fixes. A group of five test users each played three separate games with instructions from the player guide in Appendix B. The set of seeds played was identical across testers to ensure consistency, allowing the results to be compared with greater precision. After playing the game, the testers filled out a short evaluation form with a series of questions regarding each aspect of the game.

The form was designed in particular to evaluate the game based on its primary and secondary requirements. In addition, the questions reflect the design goals of a sense of flow, excitement and improvement, as well as the aim of playable, varied and consistent procedural generation. The Likert scale [23] was used to measure their experiences. There was also a space for additional comments and explanation of any issues encountered. The evaluation adhered to the ethical considerations outlined before, and therefore did not collect any personal information.

Figure 24 provides an overview of the testing results. The complete survey data can be found in Appendix C. Overall, the players enjoyed the game and rated it positively, expressing a desire to try future versions. The clarity and function of the menus and UI were unanimously positively rated, as was the visual and auditory appeal. In particular, testers mentioned that they liked how the game committed to its retro aesthetic, which reminded them of older games they had played. Personalising the game with the different colour palettes was also a popular feature.

Regarding the gameplay, for most players it elicited the sense of excitement and challenge that the design was centred around, though not all elements were equally engaging. Players disagreed whether the regular or special attack was more powerful, which suggests a decent balance, but were not sufficiently incentivised to use them in conjunction. Furthermore, players neglected the streak system somewhat – likely because the streak and its effects were not communicated strongly enough. Nevertheless, most testers found the combat satisfying and especially enjoyed trying the various powers.

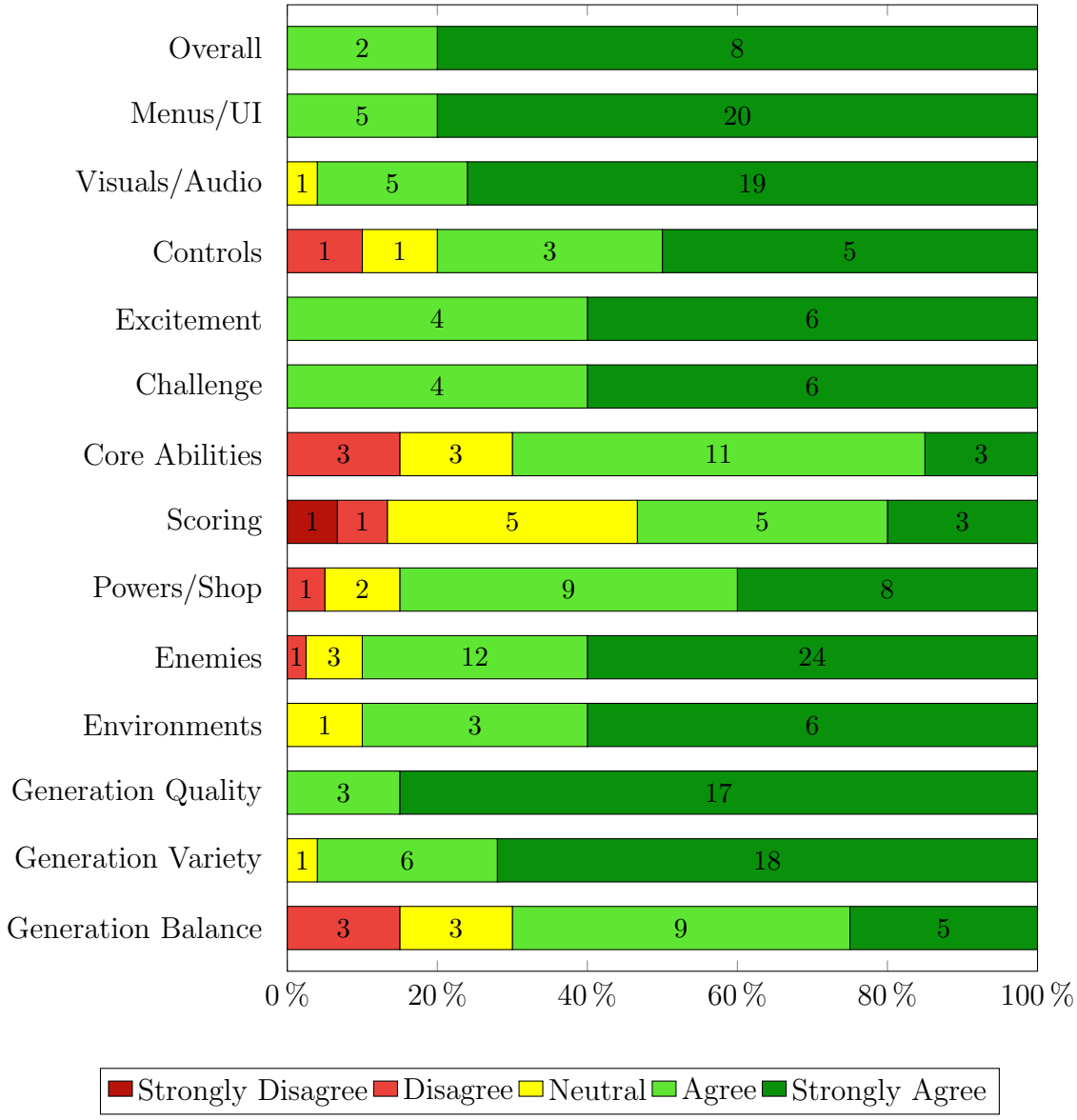


Figure 24: An overview of playtest feedback responses on the Likert scale, aggregated into categories.

For the procedural generation, players were generally very impressed by the variety and sprite design of the enemies. The pacing and layout of the waves was well rated as well. However, there were still some balancing issues, as players did find the occasional enemy that was far too powerful. The designs were also not always clear about the power level of the enemies, as a single smaller highlight could indicate the same power level as a single larger highlight. The environments, meanwhile, were found to be interesting in terms of gameplay, but did not change remarkably between levels. In general, the three generated games were rated as equally fun and varied, but not equally difficult.

Several changes were made as a result of this feedback, in addition to the fixing of reported bugs. The balance of certain powers and generated enemies was adjusted to reduce outliers in effectiveness. The streak system was given additional visual and sound cues, such as a halo around the player and a change in projectile appearance, to make the boosted state feel more significant. Finally, a few small quality of life features were added that were requested multiple times, including a controls screen and the ability to pause.

8.2 Requirement Fulfilment

The final evaluation of the project is based on the extent to which the initial primary and secondary requirements were met. The more subjective targets of a fun, varied player experience and a tense, exciting feeling are also crucial considerations here.

8.2.1 Primary Requirements

The primary requirements of the project were all fulfilled to a satisfactory extent, so the game was an overall success. There remains room for expansion and refinement, however.

1. *Create a playable shoot 'em up framework with essential game mechanics, including a level system, player movement and shooting, as well as enemy and environment interaction.*

   All essential components of a shoot 'em up are present in the game. Testing and user feedback indicates that the mechanics generally function as intended with few bugs, and that the core systems are fun and intuitive for players.

2. *Develop a procedural generation algorithm for levels populated with environmental features and waves of enemies.*

   The multi-step environment and background generation fulfils this requirement, producing interesting terrain that fits the game aesthetics. However, based on feedback, the variety would be significantly improved if more types of procedural shapes were implemented that would alternate between levels. The impact of the environments on gameplay could also be improved through added collectibles and hazards.

3. *Develop a procedural generation algorithm for the visuals, movement and attacks of enemies.*

   The enemy generation was highly successful, creating an attractive variety of sprites that was praised by players. Their difficulty is reasonably balanced given the variety of capabilities that they offer. The limited randomness in the wave patterns was a good choice given the development scope, but the game would certainly benefit from more dynamic enemy paths.

4. *Test the game to ensure generated content is fair and fun for players.*

   The game was tested extensively by myself to tune the pacing, challenge and feel of the game, as well as eliminate any unforeseen issues. The user feedback provided highly valuable perspectives from people who were not familiar with the game, and resulted in several important changes.

8.2.2 Secondary Requirements

Some of the secondary objectives were met, while others were altered to fit the scope or abandoned.

1. *Develop a procedural generation algorithm for weapons and power-ups.*

   The system for player powers was simplified to randomly select from a fixed number of powers rather than procedurally generate them. This ended up being an effective choice, as each handmade power is balanced and interesting, while still offering a good amount of variety with the different combinations. Because player tools have much higher standards of quality and uniqueness, algorithmically creating them would be an interesting challenge that would require more time and an increased overall game complexity.

2. *Develop an interesting scoring system that rewards and encourages players.*

   A decent scoring system was implemented that drives players to defeat enemies quickly and avoid taking damage as long as possible. Spending score at shops is also an interesting strategic decision. It is clear from the user testing, however, that there is potential for a more engaging system both mechanically and visually. This could also benefit other areas of the game, such as by rewarding players for using both weapons instead of relying on only one.

3. *Extend the level design algorithm to incorporate improved pacing and difficulty progression.*

   The system for level pacing and progression is simple but effective in creating tense moments but not overwhelming the player. Improvements are possible in differentiating the structure of levels.

4. *Extend the enemy design algorithm to incorporate unique abilities and bosses.*

   The enemies have a good diversity in statistics and abilities, resulting in interesting and strategic gameplay – although, of course, there is always room for new additions here. In the further development of the game, a key next step would be to add a satisfying ending point, which is present in most games after the arcade era. This would come in conjunction with adjusting the enemy creation to generate bosses.

5. *Polish the game visuals and mechanics to improve the game feel, clarity and aesthetics.*

   The game is relatively well polished, and a lot of effort was spent on small features to improve the game feel and aesthetics. There are still a few features that are missing compared to published games, such as detailed options and accessibility.

9 Conclusion

The purpose of this project was to explore the integration of procedural generation techniques in a shoot 'em up game to produce an exciting and replayable experience. Inspired by historical and design insights from the genre, the *Instar* game was developed in Unity focusing on the algorithmic generation of insect enemies and cell-like terrain. As evidenced by the playtest feedback, the game successfully delivers on the primary objectives with a solid gameplay foundation, varied enemy design, and interesting environment formations. Furthermore, it delivers well on several secondary objectives, including player and enemy abilities, difficulty progression and an overall polished feel. The project was an interesting insight into the diverse aspects of game development.

Learning and implementing all these systems to create a complete game took longer than anticipated, so the scope of the project was somewhat reduced over its development. There are a plenty of possible features and improvements for future work: a more defined structure with bosses, an ending and meta-progression; increased environment variations such as organic features created using L-systems; deeper and more refined systems for scoring and abilities. Nevertheless, the project is an apt demonstration of the potential of procedural generation for the genre – and it importantly delivers on the central player experience.

References

[1] Luke McMillan. 2011. *All of Your Base are Belong to Us? Shmups as a Source for Better Game Design*. Ph. D. Dissertation. Griffith University.

[2] Jim Whitehead. 2007. *Game Genres: Shmups*. UC Santa Cruz. Retrieved 1 November, 2024 from `https://web.archive.org/web/20130617091132/http://classes.soe.ucsc.edu/cmps080k/Winter07/lectures/shmups.pdf`

[3] Zach Keene. 2015. *Glossary of terms*. Shmups Forum. Retrieved 1 November, 2024 from `https://shmups.system11.org/viewtopic.php?t=11882`

[4] Mark J. P. Wolf. 2008. *The Video Game Explosion: A History from PONG to PlayStation and Beyond*. Greenwood Press, Westport, US.

[5] Brian Ashcraft. 2008. *Arcade Mania!: The Turbo-charged World of Japan's Game Centers*. Kodansha International, Tokyo, Japan.

[6] Bog Hog. 2022. *Bullet Hell Shmup Design 101*. Google Docs. Retrieved 1 November, 2024 from `https://docs.google.com/document/d/1iM9Fc2DsPppedlJVDYQ3g1VB5sFfilomGIYFIwJka9w`

[7] Nick Reichert. 2011. *Shmups 101: A Beginner's Guide to 2D Shooters*. Racketboy. Retrieved 1 November, 2024 from `https://racketboy.com/retro/shmups-101-a-beginners-guide-to-2d-shooters`

[8] Noor Shaker, Julian Togelius, and Mark J. Nelson. 2016. *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. Springer, Cham, Switzerland.

[9] Gillian Smith. 2015. An Analog History of Procedural Content Generation. In *Proceedings of the 10th International Conference on Foundations of Digital Games*. Society for the Advancement of the Science of Digital Games, California, USA.

[10] Davide Aversa. 2015. *Procedural Contents Generation: History and techniques used in the modern video game industry*. Retrieved 1 November, 2024 from `https://www.davideaversa.it/wp-content/uploads/2015/06/Procedural-Contents-Generation.pdf`

[11] Aaron Isaksen, Dan Gopstein, Julian Togelius, and Andy Nealen. 2018. Exploring Game Space of Minimal Action Games via Parameter Tuning and Survival Analysis. *IEEE Transactions on Games* 10, 2 (June 2018), 182–194.

[12] Jonas Freiknecht. 2020. *Procedural Content Generation for Games*. Ph. D. Dissertation. Mannheim University. Retrieved 1 November, 2024 from `https://madoc.bib.uni-mannheim.de/59000/1/Procedural%20Content%20Generation%20for%20Games.pdf`

[13] Mark Hendrikx, Sebastiaan Meijer, Joeri Velden, and Alexandru Iosup. 2013. Procedural Content Generation for Games: A Survey. *ACM Transactions on Multimedia Computing, Communications, and Applications* (02 2013).

[14] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. 2010. Search-Based Procedural Content Generation. In *Applications of Evolutionary Computation*. Springer, Berlin, Germany, 141–150.

[15] IntellivisionRevolution. 2014. *Game Spotlight River Raid*. IntellivisionRevolution. Retrieved 1 November, 2024 from `https://intellivisionrevolution.com/entries/intellivision-gameplay-videos/game-spotlight-river-raid`

[16] Katie Salen and Eric Zimmerman. 2003. *Rules of Play: Game Design Fundamentals*. MIT Press, Cambridge, USA.

[17] Jesse Schell. 2008. *The Art of Game Design: A Book of Lenses*. CRC Press, Boca Raton, USA.

[18] Richard Allan Bartle. 1990. Who Plays MUAs? *Comms Plus!* (October 1990), 18–19.

[19] Robin Hunicke, Marc Leblanc, and Robert Zubek. 2004. MDA: A Formal Approach to Game Design and Game Research. *AAAI Workshop* (2004).

[20] Robert Bridson. 2007. Fast Poisson disk sampling in arbitrary dimensions. *ACM SIGGRAPH* (August 2007).

[21] David F. Watson. 1981. Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes. *Comput. J.* 24, 2 (1981), 167–172.

[22] Mark Berg, Otfried Cheong, Marc Kreveld, and Mark Overmars. 2008. *Computational Geometry: Algorithms and Applications*. Springer, Heidelberg, Germany.

[23] Rensis Likert. 1932. A technique for the measurement of attitudes. *Archives of Psychology* 22, 140 (1932), 55.

Game Bibliography

Taito. 1978. Space Invaders. Taito, Tokyo, Japan.

Namco. 1979. Galaxian. Namco, Tokyo, Japan.

A.I. Design. 1980. Rogue. Epyx, San Francisco, US.

Konami. 1981. Scramble. Konami, Tokyo, Japan.

Midway. 1981. Gorf. Midway, Chicago, US.

Namco. 1981. Galaga. Namco, Tokyo, Japan.

Williams Electronics. 1981. Defender. Williams Electronics, Chicago, US.

Activision. 1982. River Raid. Activision, Santa Monica, US.

Sega. 1982. Zaxxon. Sega, Tokyo, Japan.

Vid Kidz. 1982. Robotron: 2084. Williams Electronics, Chicago, US.

Namco. 1983. Xevious. Namco, Tokyo, Japan.

David Braben and Ian Bell. 1984. Elite. Acornsoft, Cambridge, England.

Capcom. 1985. Commando. Capcom, Osaka, Japan.

Konami. 1985. Gradius. Konami, Tokyo, Japan.

Compile. 1986. Zanac. Pony Canyon, Tokyo, Japan.

Sega. 1986. Fantasy Zone. Sega, Tokyo, Japan.

Irem and Compile. 1987. R-Type. Irem, Osaka, Japan.

MicroProse. 1991. Sid Meier's Civilization. MicroProse, Hunt Valley, US.

Nintendo. 1993. Star Fox. Nintendo, Kyoto, Japan.

Toaplan. 1993. Batsugun. Toaplan, Tokyo, Japan.

Cave. 1995. DonPachi. Atlus, Tokyo, Japan.

Bethesda Softworks. 1996. The Elder Scrolls II: Daggerfall. Bethesda Softworks, Rockville, US.

Blizzard North. 1997. Diablo. Blizzard Entertainment, Irvine, US.

Cave. 1997. DoDonPachi. Atlus, Tokyo, Japan.

Treasure. 1998. Radiant Silvergun. Treasure, Tokyo, Japan.

Success Corporation. 2000. Psyvariar. Taito, Tokyo, Japan.

Treasure. 2001. Ikaruga. Treasure, Tokyo, Japan.

Bizarre Creations. 2003. Geometry Wars: Retro Evolved. Microsoft Game Studios, Redmond, US.

Cave. 2003. Ketsui: Kizuna Jigoku Tachi. AMI, Tokyo, Japan.

Cave. 2004. Mushihimesama. AMI, Tokyo, Japan.

Rake in Grass. 2004. Jets'n'Guns. Rake in Grass, Prague, Czechia.

Bay 12 Games. 2006. Dwarf Fortress. Bay 12 Games, Silverdale, US.

Mossmouth. 2008. Spelunky. Mossmouth, San Francisco, US.

Mojang Studios. 2009. Minecraft. Mojang Studios, Stockholm, Sweden.

Valve. 2009. Left 4 Dead 2. Valve, Bellevue, US.

Edmund McMillen and Florian Himsl. 2011. The Binding of Isaac. Headup Games, Düren, Germany.

Final Form Games. 2011. Jamestown: Legend of the Lost Colony. Final Form Games, Philadelphia, US.

Pixelnest Studio. 2015. Steredenn: Binary Stars. TyGAMES, Rennes, France.

Hello Games. 2016. No Man's Sky. Hello Games, Guildford, UK.

Motion Twin and Evil Empire. 2018. Dead Cells. Motion Twin, Bordeaux, France.

Chimeric. 2024. Nova Drift. PixelJAM, Asheville, US.

A Ethics Approval Form

UNIVERSITY OF ST ANDREWS
TEACHING AND RESEARCH ETHICS COMMITTEE (UTREC)
SCHOOL OF COMPUTER SCIENCE
ARTIFACT EVALUATION FORM

Title of project

Procedural Shmup

Name of researcher(s)

Barnabas Tanczos

Name of supervisor

Ian Miguel

Self audit has been conducted **YES** ☒ **NO** ☐

This project is covered by the ethical application CS15727.

Signature Student or Researcher

Print Name

Barnabas Tanczos

Date

26/09/2024

Signature Lead Researcher or Supervisor

Print Name

IAN MIGUEL

Date

30/9/24

B Player Guide

How to Play

Before playing, you can view the controls and credits in the menu. There is also the option to choose one of ten colour schemes. The game itself consists of a series of levels in which you must navigate the environment and defeat enemies until you reach the end. You have a limited number of health and ammunition, as shown by the bars on the left and right, respectively. Being hit by a projectile or crashing into the environment or an enemy causes you to lose health. Once your health reaches zero, the game is over. Try to get the highest total score that you can.

You can defeat enemies by firing at them, which uses one ammunition per attack, or using a special attack, which consumes all remaining ammunition. Use the dash to reload and dodge projectiles. Defeating enemies earns you score. By defeating enemies quickly in succession, you build up to a streak, as seen in the bar on the bottom of the screen. Once the streak is activated, you gain a boost of power and gain a multiplier to your score. Keep defeating enemies to maintain the streak.

Between levels, you can choose one of three powers to augment your attacks or give you a passive benefit. Additionally, you can spend score to heal, or gain more maximum health and ammunition.

Controls

- **Arrow Keys** - move / navigate (menu)
- **C** - regular attack / select (menu)
- **X** - special attack
- **Z** - dash
- **Esc** - pause

C Playtest Feedback

Question	Strongly Disagree	Disagree	Neutral	Agree	Strongly Agree
Gameplay					
The controls felt intuitive.	0	1	0	3	1
The player ship felt fluid and responsive.	0	0	1	0	4
I felt a sense of excitement playing.	0	0	0	0	5
The game was boring at times.	1	4	0	0	0
The game was challenging.	0	0	0	2	3
I feel I could improve over time.	0	0	0	2	3
Taking damage felt fair.	0	0	1	3	1
Weapons, Powers & Abilities					
Weapons were suitable to complete levels.	0	0	0	5	0
The dash ability was useful.	0	0	1	2	2
Preferred main weapon over special.	0	1	2	1	1
Ammunition felt too restrictive.	1	3	0	1	0
Main weapon was more powerful.	0	2	2	1	0
Used main and special weapon equally.	0	2	2	1	0
The streak system was intuitive.	1	0	1	2	1
Easy to gain/maintain streak.	0	1	2	2	0
Gaining streak was worth it.	0	0	2	1	2
Powers choice was interesting.	0	0	0	1	4
Powers affected gameplay meaningfully.	0	0	0	3	2
Shop choices were interesting.	0	0	0	4	1
Shop prices felt appropriate.	0	1	2	1	1

Question	Strongly Disagree	Disagree	Neutral	Agree	Strongly Agree
Procedural Generation					
Good variety of enemy designs.	0	0	0	1	4
Variety in enemy types and abilities.	0	0	0	0	5
Enemy designs were visually appealing.	0	0	0	0	5
Could differentiate enemies easily.	0	0	0	1	4
Difficulty progression matched powers.	0	0	1	3	1
Some enemies were too powerful.	1	1	2	1	0
Some enemies were too weak.	1	4	0	0	0
Good variety in enemy waves/formations.	0	0	0	2	3
Environment impacted gameplay.	0	0	0	1	4
Environment was visually varied.	0	0	1	2	2
Game was different between restarts.	0	0	0	1	4
Game equally enjoyable on restarts.	0	0	0	1	4
Difficulty was consistent across restarts.	0	2	1	2	0
Visuals & Audio					
The game has appealing visuals.	0	0	0	0	5
Music and sound fit the game.	0	0	0	1	4
On-screen action was clear.	0	0	0	2	3
Could distinguish stronger enemies.	0	0	1	1	3
Could distinguish player/enemy attacks.	0	0	0	1	4
UI					
Menus were easy to navigate.	0	0	0	0	5
Menus communicated sufficient info.	0	0	0	2	3
Gameplay UI provided sufficient info.	0	0	0	1	4
Gameplay UI was visually clear.	0	0	0	1	4
Icons communicated purpose clearly.	0	0	0	1	4
Overall					
I enjoyed playing the game.	0	0	0	1	4
I would play this game again.	0	0	0	1	4